AGLC CRM: Quick Reference Guide

For: Developers, Operators, Support Team
Date: March 11, 2026
Version: 1.0

🚀 Quick Start

Local Development

```
# Clone and setup
git clone https://github.com/yourorg/aglc-crm.git
cd aglc-crm
pnpm install
pnpm db:push

# Start development
pnpm dev

# Run tests
pnpm test

# Build for production
pnpm build
```

Common Commands

```
# Database
pnpm db:push              # Apply migrations
pnpm db:studio            # Open database UI

# Development
pnpm dev                  # Start dev server
pnpm test                 # Run tests
pnpm format               # Format code
pnpm check                # Type check

# Production
pnpm build                # Build for production
pnpm start                # Start production server
```

📊 System Architecture at a Glance

```
Frontend (React 19)
    ↓ (tRPC)
Backend (Express 4)
    ↓ (SQL)
Database (PostgreSQL)
    ↓ (APIs)
External Services (OpenAI, Twilio, SendGrid, S3)
```

🗄️ Database Quick Reference

Core Tables

Table	Purpose	Key Fields
`users`	Authentication & profiles	`openId`, `role`, `email`
`clients`	Client information	`name`, `email`, `vipStatus`
`cases`	Legal cases	`clientId`, `caseType`, `status`
`phases`	Case phases	`caseId`, `phaseNumber`, `status`
`tasks`	Individual tasks	`phaseId`, `assignedTo`, `deadline`
`communications`	Client interactions	`clientId`, `type`, `sentiment`
`performance`	Team metrics	`userId`, `tasksCompleted`, `qualityScore`

Common Queries

```sql
-- Get active cases
SELECT * FROM cases WHERE status = 'Active';

-- Get overdue tasks
SELECT * FROM tasks WHERE deadline < NOW() AND status != 'Completed';

-- Get team performance
SELECT userId, COUNT(*) as tasks_completed, AVG(qualityScore) as avg_quality
FROM performance
GROUP BY userId
ORDER BY tasks_completed DESC;

-- Get client communication history
SELECT * FROM communications
WHERE clientId = ?
ORDER BY timestamp DESC;
```

🔌 API Quick Reference

tRPC Procedures

```
// Queries (read-only)
trpc.cases.list.useQuery()
trpc.tasks.list.useQuery()
trpc.clients.getById.useQuery({ id: 1 })
trpc.dashboard.getExecutiveOverview.useQuery()

// Mutations (write)
trpc.cases.create.useMutation()
trpc.tasks.update.useMutation()
trpc.tasks.assign.useMutation()
trpc.clients.updatePersona.useMutation()

// Authentication
trpc.auth.me.useQuery()
trpc.auth.logout.useMutation()
```

External APIs

```
// LLM (OpenAI)
invokeLLM({ messages: [...] })

// WhatsApp (Twilio)
client.messages.create({ body, from, to })

// Email (SendGrid)
sgMail.send({ to, from, subject, html })

// File Storage (S3)
storagePut(key, buffer, mimeType)
```

🤖 AI Agents Quick Reference

Agent Triggers

Agent	Trigger	Frequency
Planner	Case created	Once per case
Tracker	Task deadline approaching	Every 15 minutes
Executor	Document uploaded	On demand
Crisis Manager	Risk detected	Every 6 hours
Personality	Task assignment needed	On demand
Consultant	Monthly analysis	Monthly

Agent Status Check

```
# View agent logs
SELECT * FROM agentLogs
WHERE agentName = 'planner'
ORDER BY timestamp DESC
LIMIT 20;

# Check agent performance
SELECT agentName, COUNT(*) as runs,
  SUM(CASE WHEN status = 'SUCCESS' THEN 1 ELSE 0 END) as successes,
  AVG(executionTime) as avg_time
FROM agentLogs
GROUP BY agentName;
```

🔐 Security Checklist

Before Deployment

- [ ] All secrets in environment variables (not in code)

- [ ] Database encryption enabled
- [ ] SSL/TLS configured
- [ ] Audit logs enabled
- [ ] Backups tested
- [ ] Security scan passed
- [ ] Rate limiting enabled
- [ ] CORS properly configured

Regular Maintenance

- [ ] Review access logs weekly
- [ ] Update dependencies monthly
- [ ] Rotate encryption keys quarterly
- [ ] Security audit quarterly
- [ ] Disaster recovery drill quarterly

🐛 Troubleshooting Quick Guide

Application Won't Start

```
# Check database connection
psql $DATABASE_URL -c "SELECT 1"

# Check environment variables
env | grep DATABASE_URL

# Check logs
tail -f .manus-logs/devserver.log
```

Slow Performance

```
# Find slow queries
SELECT query, mean_time FROM pg_stat_statements
ORDER BY mean_time DESC LIMIT 10;

# Check database connections
SELECT count(*) FROM pg_stat_activity;

# Check indexes
SELECT * FROM pg_indexes WHERE tablename = 'cases';
```

AI Agent Not Working

```
# Check agent logs
SELECT * FROM agentLogs WHERE agentName = 'planner'
ORDER BY timestamp DESC LIMIT 10;

# Check for errors
SELECT * FROM agentLogs WHERE status = 'FAILURE'
ORDER BY timestamp DESC LIMIT 10;

# Manually trigger agent
-- Call agent procedure directly
```

WhatsApp Integration Issues

```
# Check Twilio credentials
echo $TWILIO_ACCOUNT_SID
echo $TWILIO_AUTH_TOKEN

# Check webhook logs
SELECT * FROM webhookLogs WHERE service = 'twilio'
ORDER BY timestamp DESC;

# Test webhook with curl
curl -X POST http://localhost:3000/api/webhooks/whatsapp \
  -H "Content-Type: application/json" \
  -d '{"From":"whatsapp:+1234567890","Body":"Test"}'
```

📈 Monitoring Dashboard

Key Metrics to Watch

```
System Health:
  - Uptime: Should be >99.9%
  - Error Rate: Should be <0.1%
  - Response Time: p95 <200ms
  - Database Connections: <80% of max

Business Metrics:
  - Active Cases: Trend over time
  - Task Completion Rate: Should be >95%
  - Client Satisfaction: NPS >50
  - Team Utilization: >80% billable

Performance:
  - API Response Time: <200ms
  - Database Query Time: <100ms
  - Page Load Time: <3s
  - Memory Usage: <80% of available
```

View Metrics

```
# System health
curl http://localhost:3000/api/health

# Database stats
SELECT
  (SELECT count(*) FROM cases) as total_cases,
  (SELECT count(*) FROM tasks) as total_tasks,
  (SELECT count(*) FROM users) as total_users;

# Recent errors
SELECT * FROM errors ORDER BY timestamp DESC LIMIT 20;
```

🚢 Deployment Checklist

Pre-Deployment

- [ ] All tests passing (`pnpm test`)
- [ ] Code review completed
- [ ] Database migrations tested
- [ ] Environment variables set
- [ ] Secrets configured
- [ ] Backups created
- [ ] Rollback plan documented

Deployment Steps

1. Create checkpoint in Manus UI
2. Run `pnpm build`
3. Click “Publish” in Manus UI
4. Monitor deployment logs
5. Run smoke tests

6. Verify all features working
7. Check error logs for issues

Post-Deployment

- [ ] Verify application running
- [ ] Check error logs
- [ ] Monitor performance metrics
- [ ] Verify all integrations working
- [ ] Notify team of deployment
- [ ] Document any issues

📞 Support & Escalation

Contact Information

Role	Channel	Response Time
Support Team	Slack #aglc-crm-support	1 hour
Dev Team	Slack #aglc-crm-dev	2 hours
On-Call	Phone +966-XX-XXXX-XXXX	15 minutes

Escalation Levels

1. **Level 1:** Support team (basic troubleshooting)
2. **Level 2:** Dev team (code issues)
3. **Level 3:** Architecture team (system design)
4. **Level 4:** Executive team (business decisions)

Critical Issues

- System down: Page on-call immediately

- Data loss: Escalate to Level 3
- Security breach: Escalate to Level 4
- Major bug: Escalate to Level 2

📚 Documentation Links

- **Full Implementation Guide:** `AGLC_CRM_Implementation_Guide.pdf`
- **Recommendations & Evaluation:** `AGLC_CRM_Recommendations_Summary.pdf`
- **Knowledge Transfer Guide:** `AGLC_CRM_Knowledge_Transfer_Guide.pdf`
- **Architecture Diagrams:** See visual diagrams
- **API Documentation:** Auto-generated from tRPC
- **Database Schema:** `drizzle/schema.ts`

🎯 Key Contacts

Role	Name	Email	Phone
Project Lead	[Name]	[email]	[phone]
Tech Lead	[Name]	[email]	[phone]
DevOps	[Name]	[email]	[phone]
Product Manager	[Name]	[email]	[phone]
Support Lead	[Name]	[email]	[phone]

💡 Pro Tips

1. **Always run tests before committing:** `pnpm test`
2. **Use type safety:** Never use `any` type
3. **Document your changes:** Update README and docs

4. **Monitor logs regularly:** Check for errors and warnings
5. **Keep dependencies updated:** Run `pnpm outdated` monthly
6. **Backup before major changes:** Create checkpoint in Manus UI
7. **Test in staging first:** Never deploy directly to production
8. **Communicate with team:** Update Slack channel on deployments

---

🔗 Useful Links

- **GitHub Repository:** https://github.com/yourorg/aglc-crm
- **Manus Platform:** https://manus.im
- **PostgreSQL Docs:** https://www.postgresql.org/docs/
- **React Docs:** https://react.dev
- **tRPC Docs:** https://trpc.io
- **TypeScript Docs:** https://www.typescriptlang.org/docs/

Last Updated: March 11, 2026
Next Review: June 11, 2026
Maintained By: Development Team